CertaRig capstone connected rig workflow

Implementation and validation checkpoint

Draft for student and supervisor review

Working milestone 20 November 2026

Field	Recorded value
Student	Chintan Dedhia
Roll number	2023EB03005
Program	BSc Computer Science Online Mode
Institution	BITS Pilani
Academic year	Awaiting student confirmation
Supervisor	Professor Raj Kumar from prior Study Project; current assignment to confirm
Report date	9 November 2026
Evidence scope	Local simulator and deterministic fixtures

This draft is not a final university submission. The historical Phase 3 installation remains unchanged.

Declaration

Student review and signature pending. The supplied originality declaration must be checked and signed by the student before final submission. No signature or originality attestation is made by this draft.

Abstract

CertaRig is a test-operations platform that separates AI-assisted interpretation from deterministic execution and evidence recording. The Study Project established a mapped, low-voltage dry bench with pressure and flow emulators, emergency-stop handling and a controlled procedure path. The capstone extends that foundation toward reuse on another described rig. This checkpoint implements one local workflow: configure a thermal simulator, review its readiness, approve a named test, execute it and export evidence for independent review.

The implementation reuses the existing Python Edge service, browser-based operator console, configuration contracts and procedure library. It adds configuration-bound commissioning records, durable approval history, restart invalidation and propagation of approval identifiers through the agent and client APIs. Individual-run exports now contain the referenced raw recording together with configuration, procedure revision, commissioning review, approval history, events and checksums. An offline verifier recalculates the displayed sample summary from the CSV and rejects missing or corrupted evidence.

Validation combines regression tests, targeted acceptance tests, state-machine and scenario tests, a simulator soak and browser execution. The new acceptance tests exercise denial, expiry, replay, requester mismatch, configuration changes, stale observations and provider failure. The demonstrated thermal run uses simulated data and an explicitly labelled deterministic provider fixture; it is not live-model or physical-equipment qualification. The checkpoint establishes a reviewable software workflow while preserving the earlier physical submission. Live providers, real network connectors, independent user identities and new physical commissioning remain subsequent work.

Table of Contents

Chapter 1 Introduction 4

Chapter 2 Implementation Details 7

Chapter 3 Testing Validation and Results 10

Chapter 4 Execution and Deployment Details 13

Chapter 5 Project Execution Evidence 14

Chapter 6 Conclusion and Future Work 15

References

Appendices A to D

List of Figures

Figure 1 Implemented architecture and evidence flow

Figure 2 Explicit simulator approval in the operator console

Figure 3 Completed thermal simulator procedure

Figure 4 Captured local commit history

List of Tables

Table 1 Capstone acceptance cases

Table 2 Current validation results

Table 3 Genuine progress and planned review

Abbreviations

API Application Programming Interface; CSV Comma Separated Values; HIL Hardware in the Loop; LLM Large Language Model; UI User Interface; UTC Coordinated Universal Time

CHAPTER 1 INTRODUCTION

1.1 Project overview

Engineering test rigs combine sensors, controllers, application code, configuration and operating procedures. A result is useful only when another engineer can identify what ran, under which mapping and limits, who approved it and which measurements support the conclusion. CertaRig addresses this operating workflow by connecting an operator interface to an Edge service that performs deterministic validation, execution and recording. An assistant can translate an operator request into an available procedure, but it cannot grant authority to itself or replace numerical checks performed by the controller.

The project builds on the earlier Study Project. That work established a low-voltage Raspberry Pi bench using potentiometers as pressure and flow emulators, an ADS1115 acquisition path, an E-stop and relay/indicator behavior. Its evidence supports laboratory dry-bench claims. The capstone treats that installation as a reference and keeps its submitted files unchanged. The new checkpoint demonstrates reuse through a temperature-based simulator whose channels and limits can be changed through configuration rather than application-source edits.

1.2 Problem statement and motivation

The specific problem is the gap between defining a changed rig and producing a trustworthy, reviewable test result. A configuration file alone does not establish readiness. A simulator result does not establish physical wiring correctness. A model suggestion does not establish permission to execute. A report referring to a CSV does not establish that the recording was actually included in the exported package. These distinctions become important during handover, repeat testing and recovery after software restarts.

The initial platform already exposed procedures, approvals and evidence exports, but its approvals existed only in memory and its individual-run exporter selected files inside the run folder while raw recordings were stored elsewhere. Configuration application also needed to keep the running adapter, procedure library and browser contract aligned. This checkpoint addresses those concrete software gaps rather than claiming general autonomous machine operation. The intended user is an engineer or laboratory operator who understands a rig and needs a repeatable procedure and a defensible record.

1.3 Capstone objectives

The primary objective is to complete the sequence of configuration, readiness review, explicit approval, deterministic execution and independent evidence verification for a second rig. A temperature-based soak is selected because the repository already includes a thermal profile and procedure, allowing the checkpoint to test reuse beyond the original pressure/flow concepts without new equipment.

Supporting objectives are to persist approval history without restoring authority after restart; bind execution to the requesting principal, exact arguments, configuration and procedure revision; refuse unreviewed or invalid inputs; include raw samples in each run export; and reproduce descriptive results without a running service or model provider. Usability is evaluated through the actual operator console. The demonstration must include a deliberately blocked operation as well as a successful run, because refusal behavior is part of the implemented result.

1.4 Scope of implementation

This checkpoint is a local software implementation. The simulator generates observations; it does not measure a heated physical process. The AI path uses deterministic fixtures and makes no paid API calls. The existing reference bench is preserved but is not redeployed, rewired or remotely actuated. The implementation therefore makes no new claims about hydraulic performance, electrical relay timing, physical calibration, field reliability or industrial certification.

The new enforcement is enabled by a commissioning-required profile. Legacy profiles retain their existing entry points, while the capstone profile requires a reviewed configuration and an explicit grant before each run. The operator interface supports previewing and applying simulator mappings, reviewing a specific procedure, deciding an approval and downloading one run. Real MQTT integration, a Python connector package, independent user accounts, VPN access, unattended physical operation and live-provider qualification remain later increments. The broader connected-rig specification supplies direction without being represented as completed functionality.

1.5 Organization of the report

Chapter 2 describes the implemented architecture, technologies, modules and key logic. Chapter 3 records testing strategy, acceptance cases and observed results, separating the new simulator evidence from historical hardware evidence. Chapter 4 explains local execution and

the demonstration sequence. Chapter 5 records version-control provenance, authentic progress and the current supervisor-interaction status. Chapter 6 states achievements, limitations and the dependency order for further work. The appendices provide an operator guide, installation instructions, source references and a timed demonstration script.

The working milestone is 20 November 2026, selected during planning. The orientation material contains inconsistent years, so this report does not assert a final university deadline. Student metadata is carried forward from the prior report. The current academic year, supervisor assignment and final declaration require student confirmation before submission. All results in this checkpoint are labelled according to their source and validation status.

CHAPTER 2 IMPLEMENTATION DETAILS

2.1 System architecture and design

The browser console submits requests to a single authenticated Edge service. The service checks the current contract, commissioning status and approval before dispatching a named procedure. The local runner reads observations from the simulator through the existing hardware-adapter interface and executes commands through the deterministic guardrail. Acquisition and recording are independent of successful model inference. The evidence layer stores the run and produces a portable bundle.

The principal data flow is configuration proposal, operator review, approval request, grant or denial, procedure execution, CSV recording, bundle export and offline verification. Configuration and procedure hashes link the records. A completed run carries the configuration and approval snapshot used at its start, so later configuration edits cannot silently alter its provenance.

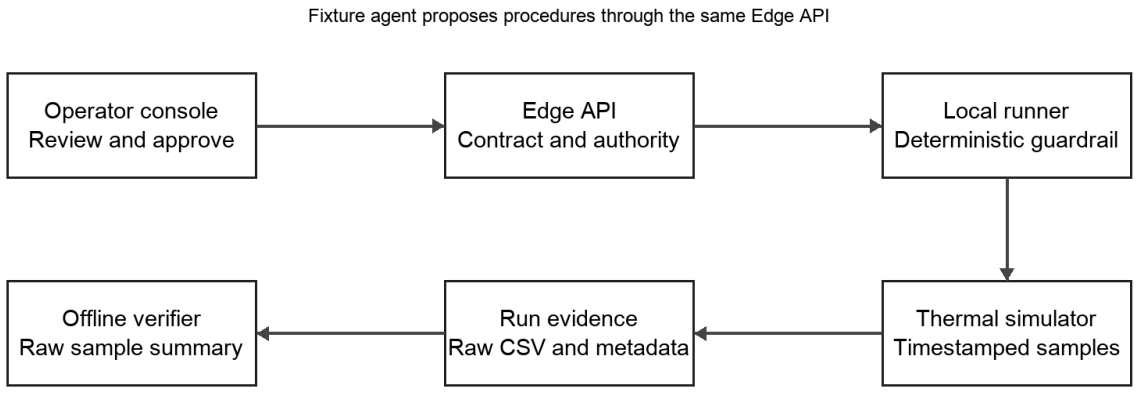


Figure 1 Implemented local architecture and evidence flow

2.2 Technology stack

The service and simulator use Python 3.11 or newer. The verified local environment used Python 3.14.3. JSON Schema validates the rig and commissioning records, YAML describes procedures, and standard-library HTTP and filesystem utilities provide the local service and evidence storage. The operator console uses existing JavaScript ES modules and browser APIs. pytest, Hypothesis, Ruff, mypy and coverage.py provide automated checks; Playwright exercises the browser workflow. No new database, cloud host or provider subscription is required for this checkpoint.

2.3 System modules

Configuration and commissioning: the existing configuration-diff path is extended with reviewed records bound to the configuration hash, adapter class and supported procedure revisions. A simulator review is labelled simulation_ready and explicitly records physical_commissioned as false. Missing channels, incompatible units, unhealthy observations and stale acquisition block readiness.

Approvals: an atomic JSON store preserves requested, granted, denied, consumed and expired history. Restart changes pending and granted records to expired. Consumption checks exact arguments, requester role/name, current contract and the procedure/adapter binding before allowing one dispatch.

Execution: the Edge client and agent tool registry forward approval_id to the procedure-start endpoint. The capstone profile requires approval for operator and agent starts. The existing runner owns the sequence and always requests the safe state at completion.

Evidence: each completed run captures configuration, procedure, commissioning and approval metadata, events, raw CSV, a deterministic report and a sample summary. Export verifies required artifacts before publication. Offline verification checks hashes and recalculates the summary from raw rows.

Operator interface: onboarding exposes a configuration editor and a procedure-specific review. The approval inbox displays the requested arguments and requester. The evidence view downloads an individual-run bundle.

2.4 Key algorithms and logic

Readiness is a deterministic conjunction of a current configuration, a compatible reviewed procedure, matching adapter identity, fresh acquisition, valid required observations and appropriate software mode. Physical commissioning is never inferred from this review. Procedure thresholds remain in the units for which the procedure was reviewed; unsupported conversions are refused.

An approval is consumed only when its stored state is granted and unexpired, its argument and contract hashes match, and its requesting role/name and revision binding match the dispatch. Restart invalidates outstanding authority regardless of whether shutdown was clean. Configuration changes invalidate grants and require a matching new review.

Export first validates the run folder, then snapshots file bytes and records their checksums. Offline analysis counts raw rows, calculates channel minima/maxima and checks the

recorded command invariant. Those descriptive results are compared with sample_analysis.json. They are reproducible summaries, not a signed certification or a complete independent reconstruction of every physical event.

Readiness → human review → exact-argument approval → one deterministic run → raw-data bundle → offline verification

2.5 Screenshots and source examples

The following figures are captured from automated browser execution of the local simulator. The browser harness clicks the human-facing controls; it is not evidence that a physical operator conducted a hardware trial. Figure 2 shows the explicit review/approval workflow. Figure 3 shows a completed thermal run. The source examples below identify the public API boundary without exposing credentials or private network addresses.

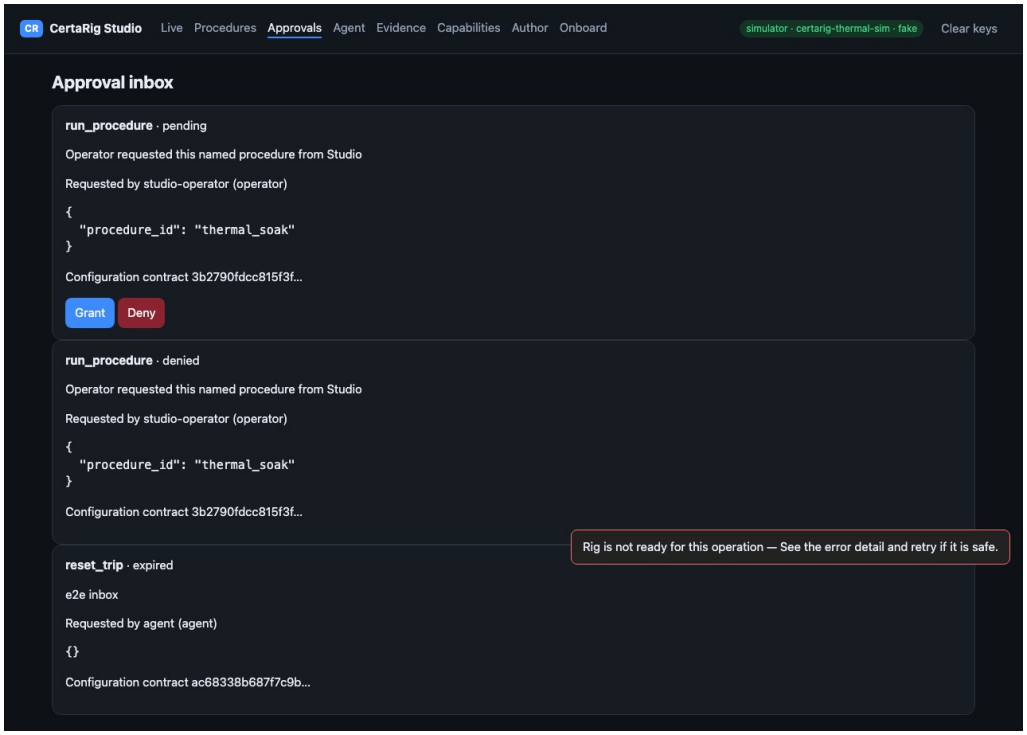


Figure 2 Browser test exercising the human approval controls

Review API: POST /v1/commissioning/review

Expected configuration hash and procedure_ids identify the review.

Execution API: POST /v1/procedure_runs

procedure_id and approval_id identify the authorized run.

CHAPTER 3 TESTING VALIDATION AND RESULTS

3.1 Test plan

Testing begins with the unmodified source baseline and continues through targeted acceptance cases, existing regression suites, property-based invariants, simulator scenarios, a separate accelerated soak and browser execution. Hardware and live-provider tests are excluded from the new software claim. The coverage threshold remains at its existing 90 percent; it was not lowered to accommodate the changes.

The baseline contained 233 passing selected software tests, with a pre-existing lint import-order error. That error was corrected in the isolated implementation branch. The final make check result is the current source validation, not a reuse of historical test counts.

3.2 Test cases

Table 1 Capstone acceptance cases

ID	Scenario	Input	Expected outcome	Status
CT01	Unreviewed rig	Start thermal soak	Readiness refusal	Pass
CT02	Missing grant	Start reviewed rig	Approval required	Pass
CT03	Happy path	Review and grant	Thermal run and verified export	Pass
CT04	Wrong requester	Another role or name	Execution refused	Pass
CT05	Revision and arguments	Changed procedure or label	Grant refused	Pass
CT06	Denial expiry replay	Denied expired or used grant	Execution refused	Pass
CT07	Restart	Pending and granted records	History retained; authority expired	Pass
CT08	Configuration edit	New channel mapping	Review/grants invalidated	Pass

ID	Scenario	Input	Expected outcome	Status
CT09	Observation faults	Missing bad or saturated sample	Readiness blocked	Pass
CT10	Units and clocks	Wrong unit or stale source	Safe refusal	Pass
CT11	Fixture agent	Named thermal request	Waits for human-facing decision	Pass
CT12	Provider failure	Fixture inference error	Recording continues; no grant	Pass
CT13	Corrupt evidence	Missing CSV or changed summary	Offline/export refusal	Pass
CT14	Unsafe storage	Corrupt store or hostile ZIP	Fail closed	Pass

3.3 Results and analysis

Table 2 Current validation results

Check	Observed result	Interpretation
make check	247 selected tests pass; lint and types pass	Software regression evidence
Coverage	90.03 percent	Existing 90 percent threshold met
Capstone acceptance	14 parameterized test cases pass	Local fixture/simulator behavior
Browser	7 tests pass	Includes configure deny grant run export
Accelerated soak	1 slow scenario passes	Simulated time; not hardware uptime

The independently verified demonstration ZIP contains 77 recorded rows, including 75 sample rows and 2 command rows. The jacket-temperature values range from 47.41051 to 47.66502 degrees Celsius. The offline calculation found 0 recorded unsafe-command rows. These are simulator observations and recorded command states; they are not electrical output measurements. The receipt records the exact run and configuration hashes.

The previous Phase 3 documentation reports six passing hardware runs, 3422 samples and 34 of 34 checks. Those figures belong to the earlier dry-bench evidence and were not reproduced in this capstone run. No measured physical latency or real-model performance improvement is inferred from the present software results.

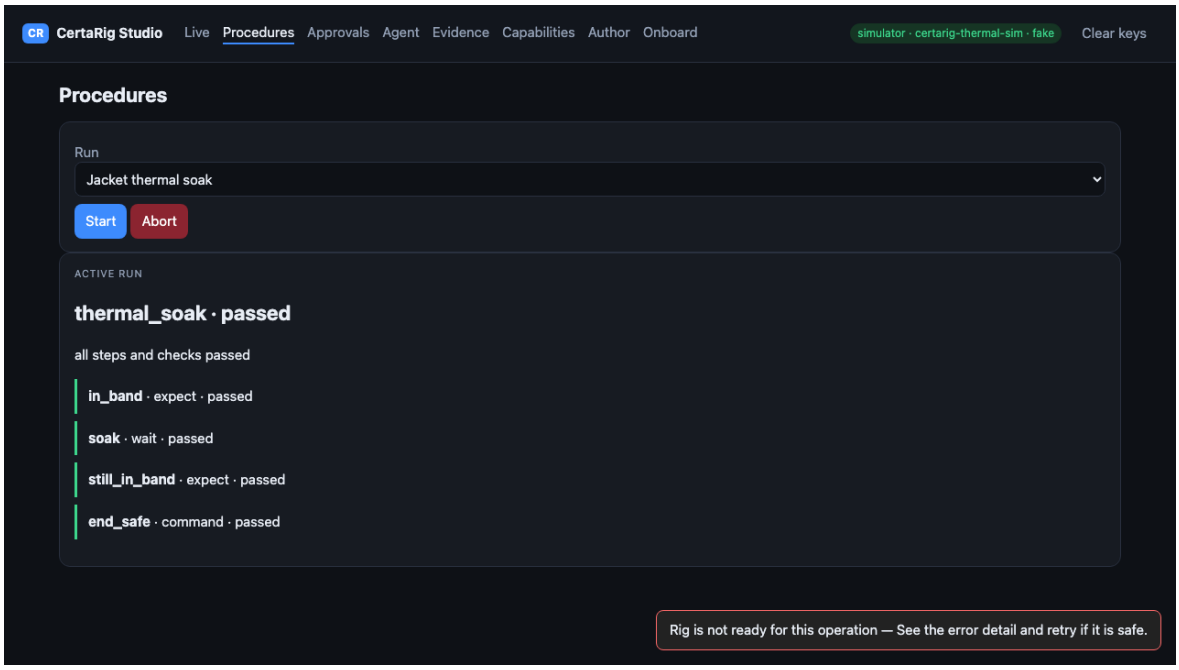


Figure 3 Completed thermal procedure in the local simulator

3.4 Failures found and corrected

Validation found a stale browser contract caused by historical run data replacing the current configuration token, duplicate onboarding rendering that could overwrite edits, and a run-display race that could expose a previous run ID during a new start. The fixes restrict token updates to current configuration endpoints and protect the affected rendering paths. The final browser suite passes after these corrections.

An early full check also found that the OpenAPI snapshot had not yet included the commissioning routes and coverage was just below the unchanged threshold. The snapshot was regenerated and meaningful failure-case tests were added. The final result is reported above; earlier failing checkpoints are not represented as successful runs.

CHAPTER 4 EXECUTION AND DEPLOYMENT DETAILS

4.1 Execution environment

The application runs locally on the developer workstation with a loopback HTTP service. The supplied capstone configuration selects the thermal simulator and the capstone capability manifest requires human approval for procedure runs. All new demonstrations use simulated observations. The historical Raspberry Pi installation is outside this checkpoint's execution environment.

4.2 Installation and deployment steps

Create the Python environment with make install. Copy the capstone rig configuration into a writable runtime directory and launch the simulator service using the command in Appendix B. Open the loopback URL, sign in with the documented simulator demo key and leave the provider set to fake. The copied runtime configuration avoids changing tracked examples. No deployment to hardware or public hosting is part of this procedure.

4.3 Demonstration sequence

First change the thermal channel mapping through Onboard and apply the previewed configuration. Attempting to start before review must be refused. Review the current configuration for the Jacket thermal soak procedure. Request a run, deny the first approval, then request again and explicitly grant it. Show the passed procedure and export that individual run. Verify the ZIP in a separate process after stopping the service, then demonstrate that a damaged copy fails verification.

The slide deck includes a five-minute live-demo allocation within a fifteen-minute run-of-show. Browser execution validates the software sequence. A spoken student rehearsal and the live viva remain pending; a timing script is not evidence that those human activities occurred.

4.4 Demo artifacts and video status

The checkpoint pack includes screenshots, a reproducible simulator evidence ZIP, a JSON receipt and an editable ten-slide deck. No new narrated capstone video or publicly hosted demo link is claimed. The prior Phase 3 film is historical background only. If the course requires a new video link, record the approved script after student review and add its actual link to the final report.

CHAPTER 5 PROJECT EXECUTION EVIDENCE

5.1 Version control evidence

The implementation starts at d81c3c099b79a9cf86ad1d7ecec7b96e333f84c2 on the platform repository and uses the isolated branch capstone/connected-workflow-2026-11-09. The original research checkout and frozen course installation were preserved. Reviewable local commits and a patch accompany the handover. No remote publication, merge or university submission is claimed.

Repository reference: https://github.com/shadybrook/certarig-platform. The local handover receipt records the final implementation commit. A generated history image in the evidence pack records the actual local commit history; it is not a screenshot of a published GitHub branch.



Figure 4 Captured local commit history rendered for review

5.2 Weekly progress summary

Table 3 Genuine progress and planned review

Week	Planned task	Recorded work	Supervisor remark
9–15 November	Implement workflow and obtain review	Implementation and validation completed 9 November; supervisor review planned	
16–20 November	Address feedback rehearse and hand over	Planned; no completion claimed	

5.3 Supervisor interaction summary

The prior report records a scope review on 7 September and a demo brief on 16 September 2026. These are historical records, not capstone meetings. No new supervisor response or attendance is verified. The unsent update seeks confirmation of the current assignment, scope, academic year and final schedule.

CHAPTER 6 CONCLUSION AND FUTURE WORK

6.1 Summary of implementation

The checkpoint implements one complete local workflow for a second configured rig. Commissioning reviews, durable approvals, per-run raw-data export and offline verification now connect the existing operator UI, agent tools and deterministic runner. The thermal mapping is changed without application-source edits, demonstrating software reuse beyond the earlier pressure/flow profile.

6.2 Achievements

The implementation preserves historical evidence, records denied and expired authority, prevents reuse of consumed grants, invalidates outstanding authority after restart and exports the raw samples necessary to reproduce the displayed sample summary. Regression and browser checks exercise both successful behavior and deliberate refusal. Evidence labels distinguish simulated observations, fixture decisions and command-derived output state.

6.3 Limitations

This is not a completed industrial platform. The thermal plant is a generic, physics-lite simulator and does not validate a real heated process. Live-model performance, physical commissioning, electrical feedback, remote operation and real broker-connected MQTT remain unqualified. The current authentication model uses shared role credentials and caller-supplied display names; it does not establish independent person-level identities. Checksums establish internal consistency rather than external authenticity.

The academic pack is a draft. No new supervisor feedback, scope approval, attendance, final deadline or signed originality declaration is invented. A new capstone video and a human spoken rehearsal still require the student. Historical hardware results are cited separately and were not rerun here.

6.4 Future enhancements

Continue in dependency order: complete shared Python and real MQTT connector contracts; qualify one live provider against routing, approval, refusal and replay cases under an agreed access/spend limit; add independent authenticated identities, rig-specific roles and recovery controls for a private remote pilot; and only then perform separately authorized physical commissioning. A protected water-loop or other customer rig is a possible later validation case, rather than the definition of the software product.

REFERENCES

[1] CertaRig platform source, base commit d81c3c099b79a9cf86ad1d7ecec7b96e333f84c2, local capstone implementation branch, inspected 9 November 2026. https://github.com/shadybrook/certarig-platform

[2] CertaRig Phase 3 Implementation Readiness and Validation, prior report, deliverables/phase3-document/CertaRig_Phase3_Document.md, September 2026. Historical hardware claims only.

[3] CertaRig Connected Rig Pilot Implementation Specification, docs/product/CONNECTED_RIG_PILOT_SPEC.md, 7 November 2026. Product direction; broader features remain planned.

[4] Capstone Rubrics, supplied DOCX. Evaluation categories and marks.

[5] Document Format, supplied DOCX. Report chapters, required evidence and formatting.

[6] Capstone Project 15 Minute Demo Template, supplied PPTX. Ten-slide sequence and university template.

[7] Current validation receipts, validation-summary.json, demo-receipt.json and accompanying check logs, 9 November 2026. Local software evidence.

APPENDIX A USER MANUAL

Sign in to the local simulator console. In Onboard, preview and apply the desired simulator configuration. Review that exact configuration for the intended thermal procedure. In Procedures, request a run. In Approvals, inspect the request and either deny or grant it; granting permits one dispatch. Follow the procedure result, then choose Export this run in Evidence. Run the offline verifier on the downloaded ZIP. Force safe remains available; any failed readiness check requires investigation before a fresh request.

APPENDIX B INSTALLATION GUIDE

make install

mkdir -p work/capstone-local

cp config/rig.thermal.capstone.json work/capstone-local/rig.json

.venv/bin/python -m certarig sim serve --port 18090

--config work/capstone-local/rig.json

--capabilities config/capabilities.capstone.json

--evidence-dir work/capstone-local/evidence

The last four lines form one shell command; join the option lines to the simulator command. Open http://127.0.0.1:18090. Demo credentials are documented by the simulator CLI and must not be used for remote hosting.

APPENDIX C SOURCE AND EVIDENCE

Platform repository: https://github.com/shadybrook/certarig-platform. New implementation is local until explicitly published. See the handover receipt, code patch, simulator evidence ZIP and validation summary in this pack.

APPENDIX D DEMO AND VIVA

New capstone video link: pending student recording and approved hosting. Historical Phase 3 film is not a substitute for the current checkpoint.

Slide	Time	Purpose
1	0:00–0:30	Introduce CertaRig and identify this as the local capstone checkpoint.
2	0:30–1:30	Explain the need to connect setup, permission, execution and evidence.

Slide	Time	Purpose
3	1:30–2:30	State the complete workflow and simulated/fixture boundaries.
4	2:30–3:30	Reuse the prior Study Project and explain the concrete software gaps.
5	3:30–5:00	Explain the agent boundary, deterministic runner and evidence path.
6	5:00–5:45	Name the stack and explain why no paid provider or new infrastructure is needed.
7	5:45–10:45	Configure, block, review, deny, approve, run, export and verify.
8	10:45–12:15	Show current test receipts and reproduce raw sample counts/extrema.
9	12:15–13:30	Discuss restart recovery, UI races, missing artifacts and unqualified physical claims.
10	13:30–15:00	State demonstrated capability and the dependency order for next work.

Likely viva questions: Why does the model not control outputs directly? What does a commissioning review prove? Why do outstanding grants expire after restart? How is a grant bound to a procedure revision? Why must raw CSV be included? What can be reproduced offline? What remains unproven on physical equipment? Which evidence demonstrates actual supervisor communication?